

INTERNAL AND EXTERNAL MEMORY

EXERCISE 1 For a non-random access memory device (like a magnetic disk), the average time (T_N) to read or write 1,000 bits is 0.00055 seconds, given an average access time (T_A) of 0.0005 seconds. Determine the transfer rate (R).

The transfer rate (R) of the non-random access memory device is (20,000,000 bits per second)) (or (20Mbps)

Formula

For a non-random access memory device, the total average time (T_N) to read or write N bits is the sum of the average access time (T_A) and the data transfer time N/R .

$$T_N = T_A + N/R$$

Where:

- T_N = Total average time to read or write N bits (0.00055 seconds)
- T_A = Average access time (0.0005 seconds)
- N = Number of bits (1,000 bits)
- R = Transfer rate (in bits per second)

Step-by-Step Calculation

1. **Rearrange the formula to solve for R :**

$$T_N - T_A = N/R$$

$$R = N/(T_N - T_A)$$

2. **Substitute the given values into the equation:**

$$R = 1,000/(0.00055 - 0.0005)$$

3. **Calculate the difference in the denominator:**

$$0.00055 - 0.0005 = 0.00005 \text{ seconds}$$

4. **Divide to find R :**

$$R = 1,000/0.00005 = 20,000,000 \text{ bits/second}$$

Converting this to megabits per second $1 \text{ Mbps} = 10^6 \text{ bits/second}$:

$R = 20 \text{ Mbps}$

- - - - -
--

Exercise 2 (OVERVIEW) In secondary memory performance analysis (such as hard disk drives), the **Total Average Time (T_N)** measures the complete duration required to satisfy an I/O request. It is determined by adding the mechanical Average Access Time (T_A) to the electronic Transfer Time (T_t). Because **T_A** relies on mechanical movements—specifically **seek time (T_s)** and **rotational latency**—it represents a massive performance bottleneck compared to the data transfer itself.

Explanation:

1. Key Performance Metric Formulas

To evaluate storage performance accurately, three fundamental equations are utilized:

- **Average Rotational Latency (T_r):** The time it takes for the target disk sector to rotate beneath the read/write head. On average, this is estimated as half the time required for a full rotation:

$$T_r = \frac{1}{2} \times (60 \text{ seconds/RPM})$$

- **Average Access Time T_A :** The overhead required to physically locate the data before transmission begins:

$$T_A = \text{Seek Time } (T_s) + \text{Average Rotational Latency } (T_r)$$

- **Total Average Time (T_N):** The absolute execution time for a block of data:

$$T_N = T_A + T_t = T_A + (\text{Block Size } (B) / \text{Transfer Rate } (R))$$

- - - - -
-

EXERCISE 2 Sample Numerical Problem Set

To illustrate these formulas, assume a system with the following performance specifications:

- **Average Seek Time (T_s):** 5 ms
- **Spindle Rotational Speed:** 7200 RPM
- **Data Transfer Rate (R):** 50 MB/s (OR 50,000,000 bytes/sec)

Step A: Determine Average Access Time (T_A)

1. Find the time required for one full spindle rotation:
 $\text{Rotation Time} = 60 \text{ sec} / 7200 = 0.00833 \text{ sec} = 8.33 \text{ ms}$
2. Compute the average rotational delay (T_r):
 $T_r = 8.33 \text{ ms} / 2 = 4.17 \text{ ms}$
3. Combine seek time and rotational latency to find (T_A):
 $T_A = 5.0 \text{ ms} + 4.17 \text{ ms} = 9.17 \text{ ms}$

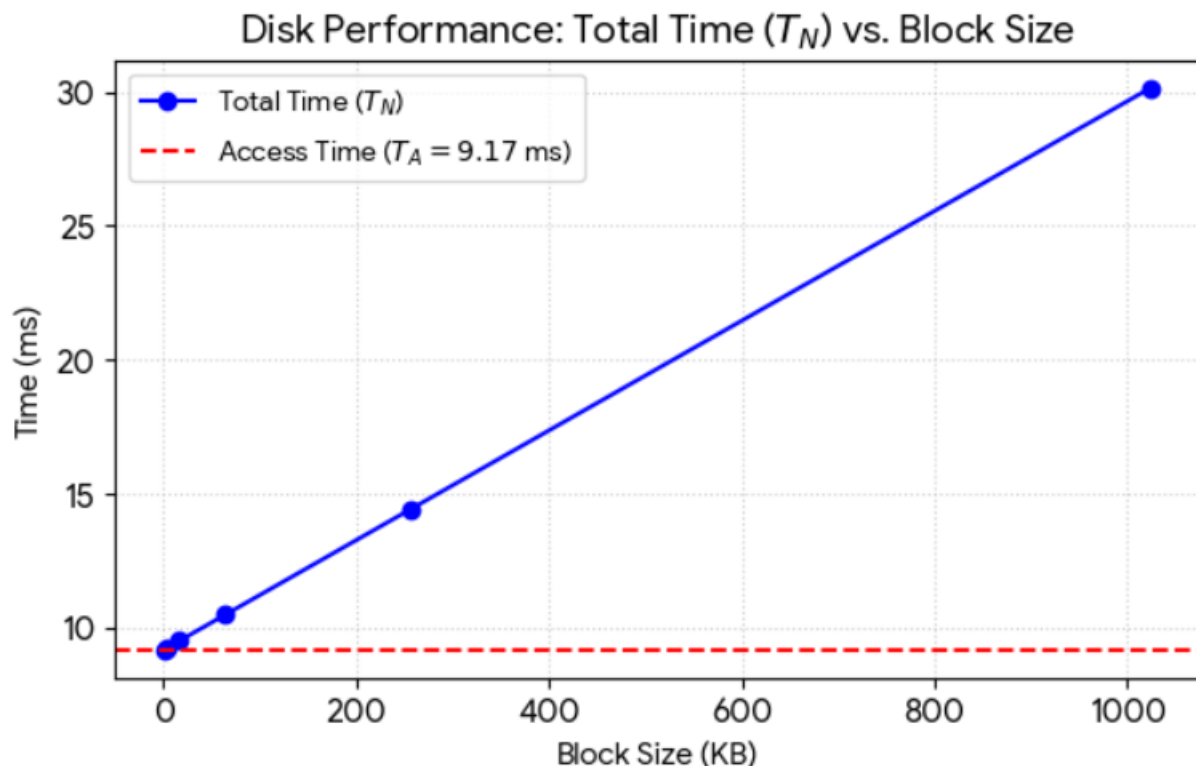
Step B: Compute Total Average Time (T_N) for Variable Block Sizes

Let's analyze two different system block sizes (B_1 and B_2) under these conditions:

- **Case 1: Small Block Size ($B_1 = 512$ bytes)**
 - **Transfer Time (T_{t1}):** (512 bytes) / (50,000,000 bytes/sec) = 0.00001024 sec = 0.01024 ms
 - **Total Time (T_{N1}):** 9.17 ms + 0.01024 ms = 9.18 ms
- **Case 2: Large Block Size ($B_2 = 4096$ bytes or 4 KB)**
 - **Transfer Time (T_{t2}):** (4096 bytes) / (50,000,000 bytes/sec) = 0.00008192 sec = 0.08192 ms
 - **Total Time (T_{N2}):** 9.17 ms + 0.08192 ms = 9.25 ms

Metric Comparison Overview

The visual behavior of T_N against T_A illustrates why operating systems prefer larger block sizes to maximize storage bandwidth efficiency:



As demonstrated by the calculation values and the graph layout above, scaling up the block size by **800%** (from 512 B to 4 KB) only increases the total handling time by a tiny fraction (0.76%). This is because the physical positioning time (T_A) heavily controls performance for small transfers.

- - - - -

EFFECTIVENESS OF MEMORY HIERARCHY

Exercise 3 (OVERVIEW) While CPU speeds have grown exponentially, the access speed of main memory (RAM) has lagged behind.

If a CPU directly accesses RAM for every instruction, it stalls—wasting most of its processing power waiting for data. The memory hierarchy solves this by layering smaller, faster, and more expensive memory close to the CPU, and larger, slower, cheaper storage further away.

Why It Works: The Principle of Locality

The hierarchy relies on **Locality of Reference**, which dictates that programs do not access data entirely at random:

- **Temporal Locality:** If a program accesses a data item once, it will likely access it again soon (e.g., loops).
- **Spatial Locality:** If a program accesses a data item, it will likely access neighboring data items soon (e.g., arrays or sequential instructions)

The Mathematical Formula: AMAT

The ultimate metric used to quantify the effectiveness of a memory hierarchy is the **Average Memory Access Time (AMAT)**. For a standard system with a Level 1 (L1) Cache, Level 2 (L2) Cache, and Main Memory (RAM), the formula is:

$$\text{AMAT} = \text{Hit Time (L1)} + \text{Miss Rate(L1)} \times \text{Miss Penalty(L1)}$$

Where the L1 Miss Penalty is the time spent checking the next level (L2):

$$\text{Miss Penalty(L1)} = \text{Hit Time(L2)} + \text{Miss Rate(L2)} \times \text{Access Time(RAM)}$$

SAMPLE NUMERICAL PROBLEM and VERIFICATION

EXERCISE 3 Scenario A: System WITHOUT a Cache Hierarchy (Direct RAM Access)

Imagine a processor that must fetch every piece of data directly from the main memory.

- **CPU Clock Cycle:** 0.5 ns (equivalent to a 2 GHz processor)
- **RAM Access Latency:** 100 ns
- **AMAT without hierarchy: 100 ns**

Result: The CPU must wait **200 clock cycles** every time it needs to read or write data.

Scenario B: System WITH a Memory Hierarchy

Now, let's introduce a two-tier cache system utilizing realistic metrics:

- **L1 Cache:** Hit Time = 1 ns (2 clock cycles); Hit Rate = 90% (Miss Rate = 10% or 0.10)
- **L2 Cache:** Hit Time = 5 ns (10 clock cycles); Local Hit Rate = 80% (Miss Rate = 20% or 0.20)
- **Main Memory (RAM):** Access Latency = 100 ns

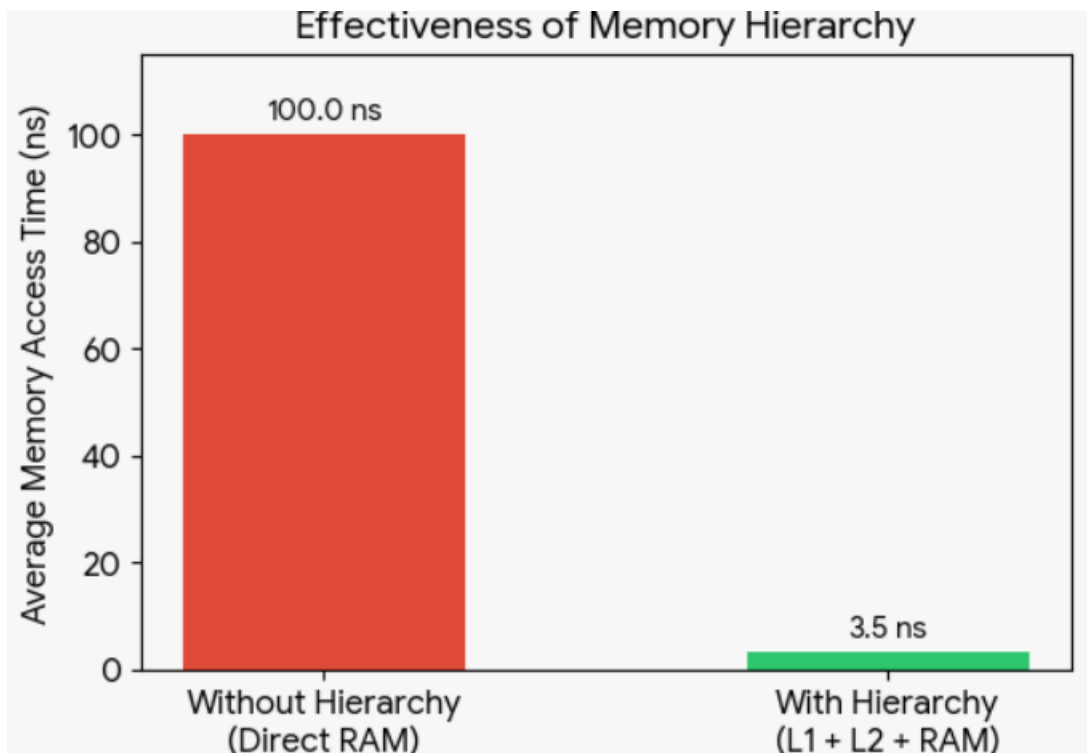
tep-by-Step Calculation:

1. **Calculate L1 Miss Penalty:**

$$\text{Miss Penalty(L1)} = 5 \text{ ns} + (0.20 \times 100 \text{ ns}) = 5 \text{ ns} + 20 \text{ ns} = 25 \text{ ns}$$

2. **Calculate Total AMAT:**

$$\text{AMAT} = 1 \text{ ns} + (0.10 \times 25 \text{ ns}) = 1 \text{ ns} + 2.5 \text{ ns} = 3.5 \text{ ns}$$



Conclusion

By implementing a memory hierarchy, the average time the CPU spends waiting for data drops dramatically from **100 ns to just 3.5 ns**.

Even though the system still relies on a slow 100 ns RAM module to store the bulk of the data, the principle of locality ensures that the CPU finds what it needs in the ultra-fast L1 and L2 caches 98% of the time ($90\% + (10\% \times 80\%)$). This delivers the **speed of the fastest memory tier** alongside the **massive capacity of the cheapest tier**

EXERCISE 4 (OVERVIEW) The effectiveness of a memory hierarchy is measured using Average Memory Access Time (AMAT). Because CPUs operate at speeds

exponentially faster than standard RAM, memory hierarchies utilize small, hyper-fast caches (like SRAM) closer to the processor to bridge this speed gap.

When a processor requests data, it searches down through the hierarchy (L1 → L2 → L3 → Main Memory). If the data is found, it is a **hit**; if not, it is a **miss**, causing a penalty as the system requests data from the layer below.

$$\text{AMAT} = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$$

Scenario 1: Base 2-Level Cache Design

Consider a standard system with an L1 cache, an L2 cache, and Main Memory. Let's assign realistic hardware cycle values:

- **L1 Cache:** Hit Time = **1 cycle**, Local Miss Rate = **5%**
- **L2 Cache:** Hit Time = **10 cycles**, Local Miss Rate = **20%**
- **Main Memory:** Access Time = **100 cycles**

To find the overall AMAT, we solve from the bottom up. First, calculate the penalty of missing L1, which depends on L2's performance:

$$\text{L1 Miss Penalty} = \text{Hit Time(L2)} + (\text{Miss Rate(L2)} \times \text{Memory Access Time})$$

$$\text{L1 Miss Penalty} = 10 + (0.20 \times 100) = 30 \text{ cycles}$$

Now, compute the final AMAT:

$$\text{AMAT} = \text{Hit Time(L1)} + (\text{Miss Rate(L1)} \times \text{L1 Miss Penalty})$$

$$\text{AMAT} = 1 + (0.05 \times 30) = 2.5 \text{ cycles}$$

Scenario 2: Changing the Cache Block Size

Cache block size dictates how much data is pulled from memory on a single miss.

- **Increasing block size** lowers the L1 miss rate because it pulls adjacent data into the cache, capitalizing on **spatial locality**.
- However, larger blocks take longer to fetch from the underlying layer, which **increases the miss penalty**.

Let's say we double the block size. The L1 local miss rate drops from 5% to **3%**, but transferring the larger block increases the L2 hit time to **12 cycles** and Main Memory access to **110 cycles**:

$$\text{New L1 Miss Penalty} = 12 + (0.20 \times 110) = 34 \text{ cycles}$$

$$\text{New AMAT} = 1 + (0.03 \times 34) = 2.02 \text{ cycles}$$

Takeaway: In this scenario, increasing the block size was effective because the benefit of reducing misses outweighed the extra transfer penalty time. (Note: If the block size is made *too large*, it can cause conflict misses and hurt performance)

Scenario 3: Adding a 3rd Level (L3) Cache

An L3 cache acts as a larger buffer between L2 and Main Memory. It is usually shared among all CPU cores. Let's return to the base L1/L2 configurations from Scenario 1, but insert an L3 cache layer:

- **L1 Cache:** Hit Time = **1 cycle**, Local Miss Rate = **5%**
- **L2 Cache:** Hit Time = **10 cycles**, Local Miss Rate = **20%**
- **L3 Cache:** Hit Time = **30 cycles**, Local Miss Rate = **50%**
- **Main Memory:** Access Time = **100 cycles**

We calculate from the bottom up (Memory -> L3 -> L2 -> L1):

1. L2 Miss Penalty (L3 AMAT):

$$\text{L2 Miss Penalty} = 30 + (0.50 \times 100) = 80 \text{ cycles}$$

2. L1 Miss Penalty (L2 AMAT):

$$\text{L1 Miss Penalty} = 10 + (0.20 \times 80) = 26 \text{ cycles}$$

3. Overall AMAT:

$$\text{AMAT} = 1 + (0.05 \times 26) = 2.30 \text{ cycles}$$

Summary of Performance Impact

Design Point	L1 Miss Penalty	Overall AMAT	Performance Tradeoff
Base 2-Level Cache	30 cycles	2.50 cycles	Standard baseline.
Larger Block Size	34 cycles	2.02 cycles	Best AMAT here. Capitalizes on spatial locality but increases lower-level delay.
3-Level (L3) Cache	26 cycles	2.30 cycles	Drops the L1 miss penalty from 30 to 26 cycles, smoothing out the cost of a memory stall.

Optional problem sets

A 24-bit physical address breaks down into a 10-bit tag, a 6-bit set index, and an 8-bit block offset for a 4-way set-associative cache with a 64 KB capacity, 16 MB main memory, and 256-byte blocks.

Problem Statement

Consider a byte-addressable computer with a main memory size of **16 MB** and a cache memory size of **64 KB**. The cache is organized as a **4-way set-associative** map with a block size of **256 bytes**

Find the number of bits required for the **tag**, **set index**, and **block offset** fields of the physical address, and determine which set a memory address like 0x42C8A4 maps to.

SOLUTION:

Step-by-Step Solution

1. Calculate Total Address Bits (Physical Address)

- Main memory size = 16 MB = 2^{24} bytes.
- Because memory is byte-addressable, the CPU physical address requires **24 bits**.

2. Calculate Block Offset Bits

- Block size = 256 bytes = 2^8 bytes.
- Offset field size = $\log_2(256) = 8$ bits.

3. Calculate Set Index Bits

- Total cache lines/blocks = Cache Size / Block Size = (64 KB) / (256 B) = 2^{16} Bytes / 2^8 bytes = $2^8 = 256$ Lines.
- The cache is 4-way set-associative, meaning each set contains 4 lines.

- Number of sets = Total Lines / Associativity = $256 / 4 = 64$ sets = 2^6 .
- Set index field size = $\log_2(64) = \mathbf{6 \text{ bits}}$

4. Calculate Tag Bits

- Total Address Bits = Tag Bits + Set Bits + Offset Bits
- $24 = \text{Tag Bits} + 6 + 8$
- Tag Bits = $24 - 14 = \mathbf{10 \text{ bits}}$

5. Address Mapping Example (0x42C8A4)

- Convert hexadecimal address 0x42C8A4 to binary to find its location fields.
- The lower bits correspond to the offset and set index.
- Extract the 6-bit set field from the address bits to find the exact target set container inside the cache framework.